

In JavaScript, you can generate a Base64 encoded string using the **built-in** `btoa()` **function** in the browser, or the `Buffer` **class** in Node.js. [DigitalOcean](#)

Here is how to implement Base64 encoding depending on your environment and data type.

Browser (Standard ASCII Text)

For standard text (Latin-1/ASCII characters), use the native `btoa()` (binary-to-ASCII) function. [DigitalOcean](#) +1

javascript

```
const text = "Hello World!";
const encoded = btoa(text);

console.log(encoded); // Output: SGVsbG8gV29ybGQh
```

Use code with caution.

Browser (Unicode & Emojis)

The `btoa()` function throws an error if the string contains Unicode characters like emojis, Arabic, or Chinese text. Use `TextEncoder` first to safely convert the string to UTF-8 bytes: [DigitalOcean](#)

javascript

```
function unicodeToBase64(str) {
  const bytes = new TextEncoder().encode(str);
  const binString = String.fromCharCode(...bytes);
  return btoa(binString);
}

const safeEncoded = unicodeToBase64("Hello 🌍, 你好");
console.log(safeEncoded); // Output: SGVsbG8g8J+MjSwg5L2g5aW9
```

Use code with caution.

Node.js Environment

In Node.js, the global `Buffer` object handles all types of text and binary data automatically: [DigitalOcean](#)

javascript

```
const text = "Hello World! 🌍";
const encoded = Buffer.from(text, 'utf-8').toString('base64');

console.log(encoded); // Output: SGVsbG8gV29ybGQhIPCfjIw=
```

Use code with caution.

Modern JavaScript (Baseline 2024+)

If you are working in a modern browser or runtime that supports the latest ECMAScript standards, you can leverage the native `Uint8Array` Base64 methods: [MDN Web Docs](#)

javascript

```
const bytes = new TextEncoder().encode("Hello World!");
const encoded = bytes.toBase64(); // Supported in modern JS environments

console.log(encoded); // Output: SGVsbG8gV29ybGQh
```

Use code with caution.

Converting Files or Images (Browser)

To encode an entire file or image from an `<input type="file">`, use the `FileReader` API: [YouTube · LearnAwesome +1](#)

javascript

```
function fileToBase64(file) {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file); // Reads file and adds the Data URL prefix
    reader.onload = () => resolve(reader.result);
    reader.onerror = error => reject(error);
  });
}
```

```
});  
}
```

Use code with caution.

Would you like help creating a function to **decode** the Base64 string back into text, or are you looking to generate **URL-safe Base64**?  CoreUI